

Entrega da Gramática

Definição da Linguagem: Gramática GLC e Exemplos

Aluno: Matheo Campanelli de Aquino Esteves
RA: 22.123.045-1

Professor: Charles Ferreira

São Bernardo do Campo, 2026

1. Introdução

Este documento apresenta a definição formal da linguagem criada para o projeto da disciplina CC6252 — Compiladores. A linguagem é imperativa, escrita em português, e suporta declaração de variáveis, estruturas de controle de fluxo, laços e operações de entrada e saída. Os programas escritos nessa linguagem utilizam a extensão `.prog`.

O compilador traduz programas escritos nessa linguagem para código na linguagem destino definida pelo professor após a entrega desta gramática.

2. Tokens da Linguagem

A tabela abaixo lista todos os tokens reconhecidos pelo analisador léxico. O reconhecimento é feito caractere a caractere, sem uso de bibliotecas de expressões regulares.

2.1 Palavras Reservadas

Token	Lexema	Descrição
PROGRAMA	programa	Início do programa
FIMPROG	fimprog	Fim do programa
INTEIRO	inteiro	Declaração de variável inteira
DECIMAL	decimal	Declaração de variável decimal (ponto flutuante)
TEXTO_TIPO	texto	Declaração de variável texto (string)
SE	se	Estrutura condicional (if)
SENAO	senao	Alternativa da condicional (else)
ENQUANTO	enquanto	Laço com pré-condição (while)
FACA	faca	Início do laço com pós-condição (do-while)
PARA	para	Laço com contador (for)
LEIA	leia	Comando de entrada de dados
ESCREVA	escreva	Comando de saída de dados
VERDADEIRO	verdadeiro	Literal booleano verdadeiro
FALSO	falso	Literal booleano falso

2.2 Literais e Identificadores

Token	Expressão Regular	Exemplo
ID	[a-zA-Z][a-zA-Z0-9_]*	contador, x1, nome_var
NUM_INT	[0-9]+	42, 100, 0
NUM_DEC	[0-9]+ . [0-9]+	3.14, 0.5, 2.0
TEXTO_LIT	"" [qualquer char exceto \n] ""	"ola mundo", "FEI"

2.3 Operadores e Delimitadores

Token	Lexema	Descrição
ATRIB	:=	Atribuição
MAIS	+	Adição
MENOS	-	Subtração
MULT	*	Multiplicação
DIV	/	Divisão
MENOR	<	Menor que
MAIOR	>	Maior que
MENOR_IG	<=	Menor ou igual a
MAIOR_IG	>=	Maior ou igual a
IGUAL	==	Igual a
DIF	!=	Diferente de
ABRE_PAR	(	Abre parêntese
FECHA_PAR	)	Fecha parêntese
ABRE_CHAVE	{	Abre bloco
FECHA_CHAVE	}	Fecha bloco
PONTO	.	Fim de comando ou declaração
PONTO_VIRG	;	Separador interno do comando para

Observação: comentários de linha iniciam com // e são ignorados pelo léxico.

3. Gramática Livre de Contexto (GLC)

A gramática foi projetada para análise descendente recursiva. Não possui recursividade à esquerda — a precedência de operadores aritméticos é garantida pela hierarquia `expr -> termo -> fator`, resultado da eliminação de recursividade à esquerda da regra `expr -> expr + termo`.

3.1 Regra Inicial e Bloco

```
prog      -> "programa" bloco "fimprog" "."

bloco     -> cmd bloco
          |  <vazio>
```

3.2 Comandos

```
cmd       -> declara | cmdAtrib | cmdSe | cmdEnquanto
          |  cmdFaca | cmdPara | cmdLeia | cmdEscreva

declara   -> tipo ID "."
tipo      -> "inteiro" | "decimal" | "texto"

cmdAtrib  -> ID ":=" expr "."
```

3.3 Estruturas de Controle

```
cmdSe     -> "se" "(" exprRel ")" "{" bloco "}" senaoOpt
senaoOpt  -> "senao" "{" bloco "}" | <vazio>

cmdEnquanto -> "enquanto" "(" exprRel ")" "{" bloco "}"

cmdFaca   -> "faca" "{" bloco "}" "enquanto" "(" exprRel ")" "."

cmdPara   -> "para" "(" ID ":=" expr ";" exprRel ";" ID ":=" expr ")" "{" bloco "}"
```

3.4 Entrada e Saída

```
cmdLeia   -> "leia" "(" ID ")" "."

cmdEscreva -> "escreva" "(" conteudo ")" "."
```

conteudo -> TEXTO_LIT | ID | expr

3.5 Expressões Aritméticas (sem recursividade à esquerda)

As regras abaixo garantem que * e / tenham precedência sobre + e -:

```
exprRel    -> expr opRel expr
opRel      -> "<" | ">" | "<=" | ">=" | "==" | "!="

expr       -> termo exprLinha
exprLinha  -> "+" termo exprLinha | "-" termo exprLinha | <vazio>

termo      -> fator termoLinha
termoLinha -> "*" fator termoLinha | "/" fator termoLinha | <vazio>

fator      -> NUM_INT | NUM_DEC | ID | "(" expr ")"
```

4. Exemplos de Código

4.1 Declaração de variáveis e atribuição

```
programa

inteiro a.
inteiro b.
decimal media.

a := 10.
b := 20.
media := (a + b) / 2.
escreva(media).

fimprog.
```

4.2 Estrutura condicional (se/senao)

```
programa

inteiro x.
inteiro y.

leia(x).
leia(y).

se (x > y) {
    escreva("X e maior").
} senao {
    escreva("Y e maior ou igual").
}

fimprog.
```

4.3 Laço enquanto (while)

```
programa

inteiro i.
```

```
inteiro soma.
```

```
i      := 1.
```

```
soma := 0.
```

```
enquanto (i <= 10) {
```

```
    soma := soma + i.
```

```
    i     := i + 1.
```

```
}
```

```
escreva("Soma de 1 a 10:").
```

```
escreva(soma).
```

```
fimprog.
```


4.4 Laço faça/enquanto (do-while)

```
programa

inteiro n.

faça {
    escreva("Digite um numero positivo:").
    leia(n).
} enquanto (n <= 0).

escreva("Voce digitou:").
escreva(n).

fimprog.
```

4.5 Laço para (for)

```
programa

inteiro i.

para (i := 1 ; i <= 5 ; i := i + 1) {
    escreva(i).
}

fimprog.
```

4.6 Programa completo

```
programa

// Declaracoes
inteiro a.
inteiro b.
inteiro resultado.
decimal media.

// Entrada
escreva("Digite o primeiro numero:").
leia(a).
escreva("Digite o segundo numero:").
```

```
leia(b).

// Operacoes
resultado := a + b * 2.
media     := (a + b) / 2.

// Condicional
se (a > b) {
    escreva("A e maior").
    resultado := a - b.
} senao {
    escreva("B e maior ou igual").
    resultado := b - a.
}

// Laco
inteiro i.
i := 0.
enquanto (i < resultado) {
    escreva(i).
    i := i + 1.
}

fimprog.
```